Verification-Triggered Reflection for Tool-Using Agents: A Pilot Comparison of Plan-and-Execute and Closed-Loop Replanning

Yizhang Cai[1]

School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China
2025111259@bupt.cn

Abstract. Agents that call external tools fail in two different ways: the plan is wrong from the start, or a call fails at execution time and the plan has to be repaired with the error in hand. This paper reports a small controlled pilot that separates the two. We run a plan-and-execute baseline and a closed-loop variant that verifies the recorded tool outputs against the question and triggers a reflection round when the check fails, on 24 auto-generated tasks whose answers are computed in Python. The task set has three families: tasks whose second call depends on the first result, tasks whose natural first call fails and must be replaced, and plain single-call tasks. The baseline answers 17 of 24 tasks correctly and the closed-loop variant 22 of 24; the entire difference sits in the failing-call family, which moves from 1 of 8 to 6 of 8. The verifier fires on 17 tasks while only 7 are genuine failures, so most of its cost is spent on tasks that needed no repair, and reflection repairs five of the seven first-pass errors. We report the accuracy, the trigger precision, and the cost per call.

Keywords: tool-using agents; planning; reflection; closed-loop control; pilot study

1 Introduction

Interleaving reasoning with acting lets a language model decide which tool to call next instead of committing to a plan up front [5]. Reflection lets the model inspect an outcome and revise [1,2], and verification prompts reduce fabricated intermediate results by checking each claim separately [4]. What is less often reported is how much of a reflection mechanism's benefit comes from tasks that genuinely need repair, and how much of its cost goes to tasks that do not.

This pilot isolates that question with a three-family task set in which the families differ only in whether the natural plan survives contact with the environment. A dependent family makes the second call's argument depend on the first call's output; a failing family presents a call that cannot succeed, so the agent must substitute one; a plain family needs a single successful call. Both conditions share the planner, the tools, and the model, so the comparison is between executing the plan once and executing it under a verifier that may trigger a repair round.

2 Related Work

Tool use is studied both as learned behaviour, where the model teaches itself when to call an API [6], and as prompting structure, where reasoning and acting alternate [5]. Deliberate planning over partial solutions [3] and self-critique without external feedback [1] address the plan side, while verbal reinforcement over episodes [2] and question-decomposition verification [4] address the repair side. Our pilot is closest to [2] and [4] in mechanism and to [5] in execution, but it measures the trigger precision of the check rather than only the final accuracy.

3 Method

Both conditions use the same four components.

Planner. Returns JSON with a list of calls, each naming a tool and its arguments. Five tools are available: `calc` for arithmetic expressions, `nth_prime`, `letters`, `temp` for a fixed city table, and `convert` for unit conversion. The tool specification is part of the prompt.

Executor. Runs the planned calls, at most four, and records one transcript line per call. A call that raises an exception is recorded as an error line rather than aborting the run.

Answerer. Receives the question and the full transcript and must answer from the recorded outputs only, ending with `ANSWER: <number>`.

Checker and reflection (closed-loop condition only). After execution, the checker sees the question and the transcript and returns JSON with a judgement and, when the judgement is negative, a list of additional calls. The reflected plan is executed, the transcript is extended, and the check repeats for at most two rounds before the answerer runs.

The baseline omits the checker: the planned calls are executed once and handed to the answerer.

4 Setup

The 24 tasks are generated programmatically with fixed seeds and their answers are computed in Python.

Dependent (8). The question asks for the temperature of a named city converted into miles at a fixed rate, so the argument of the second call is the output of the first; a plan that fixes both arguments in advance cannot be executed as written. *Failing (8).* The question names a city that is absent from the tool table and asks for a value; the agent must notice the failure and substitute a city that exists. *Plain (8).* A single call to `nth_prime` and `letters` followed by a multiplication.

Accuracy is the exact match of the final number within a tolerance of 0.02. We also record tool invocations, failed calls, reflection rounds, and model calls. Both conditions run `deepseek-v4-flash` at temperature 0 with one client per task, and every task is run once.

5 Results

Table 1. Accuracy by task family, 8 tasks per family, one run per condition.

Task family	Plan-and-execute	Closed loop with reflection
Dependent second call	8/8	8/8
Failing first call	1/8	6/8
Plain single call	8/8	8/8
Overall, n=24	17/24 (70.8%)	22/24 (91.7%)

Table 2. Cost and trigger behaviour on the same 24 tasks.

Measure	Plan-and-execute	Closed loop
Mean model calls per task	2.00	3.71
Mean tool invocations per task	1.42	2.21
Tasks on which the check fired	-	17
Tasks that genuinely failed on the first pass	7	7
First-pass errors repaired	-	5

Three observations follow from the tables. First, the dependent family is solved by ordinary planning: a planner that reasons about argument flow emits a two-step plan and the single-agent execution order handles it, so dependency alone does not create a need for repair. Second, the entire measured benefit comes from the family where the environment refuses a call: the closed loop recovers 5 of the 7 failures that the baseline left unanswered, which is the mechanism the design claims. Third, the check is imprecise. It fires on 17 of 24 tasks while 7 are genuine first-pass failures, so roughly six triggers in ten are spent confirming that a correct answer is correct, at a cost of 1.7 extra model calls per task across the set.

6 Discussion

The useful quantity in this pilot is not the headline 70.8% to 91.7% but the ratio between repair opportunities and repair attempts. A verifier that fires on every task converts a repair mechanism into a general overhead, which matters when calls are metered; a verifier that never fires cannot help. In our configuration the checker was asked a binary question about whether the recorded outputs suffice, and it defaulted to caution, which suggests that a tighter formulation - for example, verifying only when a transcript line indicates a tool error, or when the answerer's output fails a format check - would reduce

the trigger count without losing the repairs, since all five repairs started from an error line.

The result also clarifies which failures are recoverable at all. The two tasks that remained wrong under the closed loop are cases where the substitute call itself was missing from the repair list, which puts the ceiling of this mechanism at the quality of the repair proposal rather than at the detection step. A second reflection round did not recover them, which is consistent with the repair list being the limiting factor.

7 Limitations

The pilot has one model, one prompt set, one run per task, and 24 tasks, so the per-family rates rest on 8 items each and the two unrepaired failures correspond to a single repair-list omission. The tools are deterministic stubs, so no network failure, timeout, or partial result appears, and the environment returns errors as exceptions rather than as protocol-shaped responses. The checker's precision is measured against our own definition of a genuine failure, which is the baseline's first-pass outcome on the same task. These numbers are pilot numbers: they identify a mechanism and a cost profile, not a benchmark result.

8 Conclusion

On 24 generated tool-use tasks, a closed-loop planner with verification-triggered reflection answered 22 correctly against 17 for plan-and-execute, with the difference concentrated entirely in tasks whose natural first call fails (1 of 8 to 6 of 8). Reflection repaired five of seven first-pass errors. The check fired on 17 tasks to catch 7 failures, which locates the tuning problem in trigger precision rather than in detection, and the two unrepaired cases locate the remaining ceiling in the quality of the proposed repair.

References

1. A. Madaan et al., "Self-Refine: iterative refinement with self-feedback," in NeurIPS 36, 2023.
2. N. Shinn et al., "Reflexion: language agents with verbal reinforcement learning," in NeurIPS 36, 2023.
3. S. Yao et al., "Tree of Thoughts: deliberate problem solving with large language models," in NeurIPS 36, 2023, arXiv:2305.10601.
4. S. Dhuliawala et al., "Chain-of-Verification reduces hallucination in large language models," arXiv:2309.11495, 2023.
5. S. Yao et al., "ReAct: synergizing reasoning and acting in language models," in ICLR, 2023, arXiv:2210.03629.
6. T. Schick et al., "Toolformer: language models can teach themselves to use tools," in NeurIPS 36, 2023, arXiv:2302.04761.